

Umfassende Projektübersicht und Replizierungsanleitung

Diese Dokumentation liefert eine **allumfassende und detaillierte Übersicht** über das gesamte Projekt, untergliedert in vier Hauptbereiche. Ziel ist es, alle Informationen so vollständig festzuhalten, dass das Projekt in **jeder Einzelheit 1:1 nachgebaut** werden kann. *(Hinweis: Sämtliche kritischen oder markenrechtlich heiklen Elemente sind zunächst mit dokumentiert und können bei Bedarf vor einer öffentlichen Veröffentlichung entfernt werden.)*

KI **„Najika“

Beschreibung und Rolle: Die KI namens **Najika** ist das **künstliche Intelligenz-Modul** des Projekts. Sie übernimmt die Rolle eines intelligenten Assistenten bzw. Charakters innerhalb der Anwendung. Najika kann in natürlicher Sprache mit dem Nutzer interagieren, Informationen liefern und auf Aktionen im Spiel reagieren. Diese KI fungiert quasi als **virtueller Begleiter**: Sie gibt z.B. Hinweise, erzählt zur Story gehörende Inhalte oder passt ihr Verhalten an die Handlungen des Spielers an. Dadurch erhöht Najika die Immersion und Dynamik, indem sie wie ein eigenständiger Charakter erscheint.

Implementierung: Um Najika zu realisieren, wird ein **Natural Language Processing (NLP)**-System eingesetzt. Konkret basiert Najika auf einem **großen Sprachmodell (LLM)**. In der aktuellen Umsetzung nutzt das Projekt einen externen KI-Service: über eine API-Anbindung (z.B. OpenAI) werden Texteingaben des Nutzers an ein Sprachmodell gesendet und die generierte Antwort wird ausgegeben. Alternativ wäre auch ein lokal laufendes Modell möglich, doch aus Leistungsgründen und um Entwicklungsaufwand zu reduzieren, wurde die Cloud-KI-Variante gewählt. *(Hinweis: In der privaten Testversion wird ein umfangreiches Modell genutzt – dies gilt es vor einer öffentlichen Veröffentlichung ggf. zu prüfen, da z.B. API-Kosten oder Internetverbindung eine Rolle spielen.)*

Funktionen und Verhalten: Najika ist darauf trainiert, **kontextbezogene** Antworten zu geben. Sie hat eine definierte **Persönlichkeit/Backstory** gemäß der Spielwelt, sodass ihre Antworten dazu passen. Beispielsweise kann Najika die Rolle eines Mentors oder Freundes übernehmen, der den Spieler durch die Geschichte führt. Technisch wurde dies durch **System-Prompts** und **Beispieldialoge** realisiert, die dem KI-Modell vorgegeben werden, damit es konsistente Antworten in Najikas „Stimme“ liefert. Die KI kann zudem **Spielzustand** berücksichtigen: wichtige Variablen (z.B. Fortschritt des Spielers, vergangene Entscheidungen) werden bei Anfragen mitübermittelt, damit Najika darauf eingehen kann. Falls der Spieler z.B. eine bestimmte Aufgabe erfüllt hat, weiß Najika das und ändert entsprechend ihre Dialoge.

Integration ins Spiel: Die KI Najika ist nahtlos ins Handyspiel eingebunden. Der Spieler kann über eine Chat-ähnliche Schnittstelle oder kontextabhängig (z.B. bei bestimmten Ereignissen) mit Najika interagieren. Technisch gibt es dazu ein **Dialogfenster** im Spiel, in dem die vom KI-Modell generierten Texte erscheinen. Eine einfache Avatar-Grafik von Najika unterstreicht visuell, dass die Antworten von ihr stammen. Intern werden bei Spielereignissen entsprechende **API-Aufrufe** getriggert, um Najikas Reaktion zu erhalten. Zum Beispiel: Wenn der Spieler einen bestimmten Ort betritt, sendet das Spiel im Hintergrund eine Anfrage wie „Najika, erkläre dem Spieler was dieser Ort ist.“ an die KI-Engine, die Antwort (z.B. Beschreibung des Ortes) wird dann angezeigt.

Reproduzierbarkeit: Um die KI Najika nachzubauen, sind folgende Schritte essenziell:

- **KI-Plattform einrichten:** Besorge einen API-Schlüssel für den verwendeten KI-Dienst (z.B. OpenAI API für GPT-4). In den Projekteinstellungen hinterlegst du diesen Schlüssel sicher (nicht im Quelltext sichtbar). Lege Parameter fest wie Modell-ID, Temperature etc., entsprechend den bei Tests erprobten Werten.
- **Dialog-Logik implementieren:** Erstelle ein Modul im Spiel, das Nutzeranfragen und Spielereignisse entgegennimmt und an die KI-API sendet. Fange die API-Antworten ab und formatiere sie für die Ausgabe im Spiel-UI. Achte darauf, einen Kontext (Verlauf) mitzuschicken, damit Najika konsistente Antworten geben kann (z.B. den letzten Dialogverlauf in kurzer Form).
- **Prompt-Design:** Integriere einen fest hinterlegten System-Prompt, der dem KI-Modell die Rolle von Najika erklärt (inkl. Persönlichkeit, Wissen über die Spielwelt, verbotenes Verhalten etc.). Dieser Prompt stellt sicher, dass die KI immer „in-Character“ antwortet. Teste den Prompt ausgiebig und optimiere ihn, bis Najikas Ton und Wissen stimmig erscheinen.
- **Offline-Option (optional):** Falls eine Offline-KI gewünscht ist, könnte ein kleineres Modell lokal integriert werden. Dies erfordert jedoch erheblichen Aufwand in Bezug auf Laufzeit (gerade auf Mobile) und Trainingsdaten. In der aktuellen Phase wird daher die Cloud-Lösung präferiert.

Sobald diese Komponenten stehen, sollte Najika konsistent funktionieren. Teste Najika in verschiedenen Szenarien (verschiedene Fragen stellen, verschiedene Spielverläufe) und stelle sicher, dass die Antworten sinnvoll und thematisch korrekt sind. Damit ist die KI-Komponente vollständig reproduzierbar.

Gesicherter Bereich („Schwarze Mühle“ & „Digivice“)

Dieser Abschnitt umfasst den **internen bzw. geschützten Bereich** des Projekts, der unter dem Codenamen „**Schwarze Mühle**“ läuft, sowie das Konzept des „**Digivice**“. Hier werden die besonders kritischen oder fortgeschrittenen Features zusammengefasst, die nicht für jeden Benutzer offen zugänglich sind.

Gesicherter Bereich („Schwarze Mühle“)

Bedeutung und Konzept: Der Begriff „*Schwarze Mühle*“ verweist auf einen geheimnisvollen Ort der Macht und des Wissens – inspiriert von der Sage um den Zauberer Khabat ¹. Passend dazu bezeichnet die *Schwarze Mühle* im Projekt den **gesicherten Bereich**, der nur autorisierten Personen (Entwicklern/Testern) zugänglich ist. Hier werden **vertrauliche Funktionen** und **Debug-Werkzeuge** bereitgestellt, die für das Finetuning des Spiels und der KI gedacht sind. Man kann sich die *Schwarze Mühle* als eine Art „**geheime Werkstatt**“ innerhalb der Anwendung vorstellen, wo die Macher hinter den Kulissen die Fäden ziehen.

Zugangsbeschränkung: Um diesen Bereich zu betreten, ist eine **Authentifizierung** erforderlich. Aktuell ist das simpel gelöst: Beim Start der App prüft eine Routine, ob eine bestimmte **Passcode-Eingabe** erfolgt (z.B. ein vordefiniertes Passwort oder ein spezielles Benutzerkonto). Nur wenn die Anmeldedaten mit hinterlegten Entwickler-Zugangsdaten übereinstimmen, werden die Funktionen der *Schwarzen Mühle* freigeschaltet. In der Testphase genügt hierfür z.B. ein fest codiertes Passwort. Für eine spätere (ggf. öffentliche) Version würde man dieses System durch ein sicheres Login (Server-Authentifizierung, verschlüsselte Kommunikation) ersetzen, damit Unbefugte keinen Zugriff erhalten.

Funktionen im gesicherten Bereich: Innerhalb der *Schwarzen Mühle* stehen diverse **Admin- und Entwicklungswerkzeuge** zur Verfügung, um das Projekt zu steuern und zu überwachen. Dazu zählen u.a.:

- **KI-Überwachung & Feintuning:** Hier kann man Najikas Verhalten prüfen. Es werden Debug-Informationen zu den KI-Anfragen angezeigt (z.B. die Roh-Eingabe an das Modell und die erhaltene Antwort), um das Prompt-Design zu verbessern. Man kann live unterschiedliche Prompts ausprobieren oder die KI zwingen, gewisse Antworten zu geben (nützlich zum Testen von Spielsituationen).
- **Daten- und Spielerverwaltung:** In diesem Bereich lassen sich Spielstände, Nutzerprofile und ggf. Highscores einsehen und editieren. Da der Testbetrieb mit bis zu 6 Personen läuft, können die Entwickler hier z.B. den Fortschritt der Tester anpassen oder festhängen sollte. Ebenso sind Tools vorgesehen, um **Statistiken** zu sammeln (z.B. wie lange brauchte Spieler X für Aufgabe Y) – diese helfen beim Balancing.
- **Inhaltspflege:** Sämtliche Spielinhalte (Texte, Bilder, Konfigurationen) können über ein einfaches Interface angepasst werden. Beispielsweise lassen sich hier Story-Texte oder Beschreibungen ändern, ohne eine neue App-Version kompilieren zu müssen. Dies ist praktisch, um während der Testphase schnelles **Fine-Tuning** vorzunehmen (z.B. Tippfehler korrigieren oder Dialoge von Najika anpassen, falls sie unverständlich waren).
- **Debug-Menüs:** Die Schwarze Mühle bietet Buttons/Optionen, um direkt zu bestimmten Spielsituationen zu springen (Level überspringen, Belohnungen geben, etc.), was den Testaufwand reduziert. Auch ein **God-Mode** oder ähnliche Cheats für Tester sind hier aktivierbar, um alles schnell überprüfen zu können.

All diese Funktionen sind hinter der sicheren Schranke *Schwarze Mühle* verborgen, sodass normale Nutzer sie nicht sehen. Für die Reproduktion gilt: **Alle oben genannten Tools müssen nachgebaut werden**, zumindest in Grundzügen. Hierfür ist es hilfreich, im Code z.B. ein Flag für *DeveloperMode* einzubauen, das diese Menüs sichtbar macht. Dokumentiere auch das Passwort/den Zugangscode (derzeit z.B. „kr4b4t“ als Anspielung auf Krabat), damit Entwickler das im Team kennen. Vor einer öffentlichen Veröffentlichung wird dieser Zugang entweder entfernt oder durch ein robustes Auth-System ersetzt.

Digivice-Konzept

Hintergrund: Der Begriff **Digivice** stammt aus der *Digimon*-Welt und bezeichnet dort ein **digitales Gerät**, das die Verbindung zwischen einem Kind (Digiritter) und digitalen Kreaturen herstellt. Es dient im Original dazu, dem Digimon-Partner Energie zum Entwickeln (Digivolution) zu geben und andere besondere Funktionen zu erfüllen ². In neueren Geschichten gibt es sogar das Konzept eines „*Smartphone Digivice*“ – ein Handy, das als Digivice-Emulator fungiert und sowohl Digimon-Funktionen (Digitation, Tracking anderer Digiritter, etc.) als auch normale Handy-Features bietet ³.

Digivice im Projekt: Im Rahmen unseres Projekts fungiert das **Smartphone des Spielers selbst als Digivice**. Das heißt, wir greifen die Idee auf, dass das Handy mehr ist als nur Mittel zum Spielstart – es wird innerhalb der Spielwelt als **magisches Gerät** dargestellt, mit dem der Spieler interagiert. Dieses *virtuelle Digivice* verbindet die reale Welt (den Spieler mit seinem Handy) mit der digitalen Spielwelt. Konkret merkt man das an folgenden Elementen:

- **Story-Integration:** Gleich zu Beginn der Handlung erhält der Spielercharakter in der Geschichte ein „*Digivice*“. Dies wird visuell und narrativ umgesetzt, z.B. durch eine kurze Zwischensequenz: Die KI Najika überreicht (im Spiel) dem Spieler dieses mystische Gerät. In Wahrheit besitzt der Spieler natürlich schon sein Smartphone – die App tut aber so, als würde dieses nun zum

Digivice werden. Ab diesem Moment werden alle besonderen Aktionen im Spiel dem Digivice zugeschrieben.

- **Funktionen im Spiel:** Das Digivice in der Spielwelt übernimmt Funktionen analog zum Vorbild:
- Es kann als **Scanner** dienen – z.B. kann der Spieler mit der Smartphone-Kamera (falls AR vorgesehen ist) seine Umgebung scannen, um virtuelle Objekte zu finden. In der Testversion wurde etwa eine kleine AR-Komponente implementiert: Bestimmte Markierungen in der realen Welt (QR-Codes oder einfache Marker) können mit der Kamera erfasst werden, woraufhin im Spiel etwas passiert (z.B. ein versteckter Hinweis erscheint, der in der Story wichtig ist).
- Es **speichert Kreaturen oder Gegenstände:** Wenn das Spiel so etwas wie digitale Begleiter oder Items hat, werden diese im Digivice verwaltet. Ein Beispiel: Der Spieler fängt möglicherweise kleine virtuelle Wesen (analog zu Digimon). Diese werden dann im Menü „Digivice“ aufgelistet, wo man Details ansehen kann. Technisch ist das einfach das Inventar/Monster-Liste des Spielers, aber es wird im UI als Bestandteil des Digivice-Geräts präsentiert.
- **Kommunikation:** Ingame-Chats oder Meldungen laufen über das Digivice. Zum Beispiel erhält der Spieler „Nachrichten“ von Najika oder anderen NPCs auf sein Digivice (dargestellt als fiktive Messaging-App innerhalb des Spiels). So verschwimmt die Grenze zwischen der App und der Spielwelt – was auf dem echten Handy passiert, erscheint als Handlung auf dem virtuellen Digivice.
- **Spezialaktionen:** In Schlüsselmomenten der Story nutzt der Spieler das Digivice, um etwas zu bewirken – etwa ein Rätsel lösen (z.B. durch Eingabe eines Codes ins Digivice-Menü) oder um einen **“Digivolve“-ähnlichen Effekt** auszulösen, falls es digitale Kreaturen gibt. Letzteres könnte z.B. heißen: Hat der Spieler genug Energie gesammelt, kann er im Digivice-Bildschirm auf „Entwickeln“ klicken, und ein Begleiter entwickelt sich weiter (ähnlich wie ein Digimon digitiert).

Implementierungshinweise: Um das Digivice-Feature nachzubilden, sind mehrere Aspekte zu beachten:

- **Benutzeroberfläche:** Entwickle einen eigenen **UI-Bereich** im Spiel, der das Digivice repräsentiert. Das wurde im Projekt als eine Art In-App-Smartphone umgesetzt – mit einem Rahmen, der wie ein Gerät aussieht, und verschiedenen Icons (Scanner, Nachrichten, Inventar etc.). Hier solltest du darauf achten, dass dieser Bereich übersichtlich ist, da der Spieler ihn häufig nutzt. Im Prototyp gibt es z.B. ein **Hauptmenü-Icon** (ähnlich einer App-Schaltfläche) für das Digivice, das dieses virtuelle Gerät „öffnet“. Darin gibt es Tabs oder Menüpunkte für die genannten Funktionen (Scanner, Kreaturen, Nachrichten...).
- **AR-Integration (optional):** Falls du die Scan-Funktion umsetzen möchtest, nutze vorhandene **AR-Frameworks** (z.B. ARCore für Android oder ARKit für iOS, oder Unity's AR Foundation, falls Unity benutzt wird). In der Testversion wurden einfache Marker erkannt; plane hierfür Bibliotheken für **Bilderkennung** oder **QR-Code-Scan** ein. Verbinde das Scanner-Ergebnis dann mit Spielereignissen (z.B. Marker X erkannt -> löse Event Y im Spiel aus).
- **Datenhaltung:** Alle relevanten Daten (gefangene Kreaturen, gefundene Items, Nachrichtenverlauf) werden im **Speicher des Spiels** persistiert – entweder in einer lokalen Datenbank auf dem Gerät oder in Cloud/Server, je nachdem. Für den Nachbau kannst du zunächst lokale Speicherung (z.B. JSON Files oder PlayerPrefs bei Unity, SQLite DB etc.) nutzen, um es einfach zu halten. Wichtig ist, dass beim Öffnen des Digivice-Menüs immer der aktuelle Stand angezeigt wird.
- **Visuelle Gestaltung:** Um das Digivice optisch ansprechend zu machen, wurden in der aktuellen Version Designs verwendet, die an die originalen Digivices erinnern (ohne direkt geschützte Grafiken zu kopieren). Du kannst hierfür eigene Grafiken erstellen – achte aber auf das **geistige Eigentum:** Das Wort „Digivice“ und exakte Designs aus Digimon sind rechtlich geschützt. In der privaten Version war das noch zweitrangig, aber falls publiziert werden soll, erwäge eine Umbenennung (z.B. „Digi-Gerät“ o.ä.) und abgeändertes Design, um Konflikte zu vermeiden.

Zusammenfassend stellt das Digivice-Konzept eine **Brücke zwischen Spieler und Spielwelt** dar. In unserem gesicherten Bereich *Schwarze Mühle* kann man übrigens auch bestimmte Digivice-Funktionen testen oder manipulieren (z.B. alle Items freischalten). Für den Nachbau heißt das: Richte den Digivice-Bereich mit all seinen Untersystemen ein und verknüpfe sie mit der Spielmechanik. Teste ausgiebig, ob das Digivice tut, was es soll – es ist ein zentrales **Interaktions-Tool** im Spiel.

Handyspiel (Mobile Game) – Kern des Projekts

Der dritte Teil ist das **eigentliche Spiel für Smartphones**, in das die obigen Komponenten (KI Najika, Digivice, etc.) eingebettet sind. Hier wird beschrieben, wie das Spiel aufgebaut ist, welche Features es hat und wie man es **Schritt für Schritt nachbauen** kann.

Plattform und technische Basis

Das Spiel wurde als **Mobile App** konzipiert, primär für Android-Geräte (eine iOS-Portierung ist perspektivisch geplant, aber aufgrund von Testerschwierigkeiten vorerst zurückgestellt). Entwickelt wurde es mit der **Unity Engine (2021 LTS)**, da diese eine gute Grundlage für 2D/3D-Grafik, Touch-Steuerung und plattformübergreifende Bereitstellung bietet. Theoretisch könnte auch Unreal Engine genutzt werden; die Wahl fiel jedoch auf Unity wegen der vorhandenen Erfahrung im Team und der einfacheren Integration mancher Plug-ins (z.B. AR Foundation für Augmented Reality).

Projekt-Struktur: Das Spiel besteht aus mehreren **Szenen** (Unity-Terminologie) bzw. Screens: - *Startszene:* Zeigt Logos, Titel und prüft, ob man Entwickler ist (für *Schwarze Mühle* Zugang) oder normaler Spieler. Von hier aus gelangt man ins Hauptmenü. - *Hauptmenü:* Enthält Buttons für „Spiel Starten“, „Einstellungen“ und – für berechtigte Nutzer – den Zugang zur *Schwarzen Mühle*. Außerdem ist hier das Icon für das *Digivice* sichtbar, welches direkt ins Digivice-Menü führt (für Spieler innerhalb der Story erst verfügbar, sobald man das Digivice erhalten hat). - *Spielwelt (Hauptlevel):* Dies ist die wichtigste Szene – sie stellt die eigentliche Spielwelt dar, in der der Spieler seinen Charakter bewegt und Abenteuer erlebt. In unserem Projekt ist diese Welt als **2.5D-Top-Down-Map** gestaltet (eine Mischung aus 2D Grafiken und 3D-Umgebung). Hier gibt es verschiedene Orte (Wald, Dorf, die „Schwarze Mühle“ als Ort in der Story etc.), NPCs und Objekte, mit denen man interagieren kann. - *UI-Overlays:* Diverse Untermenüs und Pop-ups, z.B. für Inventar, Dialoge (die Dialogbox von Najika), Missionslog, etc., die über der Spielwelt eingeblendet werden, wenn nötig.

Grafik und Assets: Alle nötigen Grafiken und Modelle sind im Projektordner organisiert. Für den Nachbau beachte: - Figuren/NPCs: Aktuell verwenden wir einfache **2D-Avatare** für NPCs (inkl. Najikas Porträt) und einen simplen animierten **Sprite** für den Spielercharakter. Diese kannst du durch eigene Grafiken ersetzen – wichtig ist aber, die **Namenskonventionen** beizubehalten, damit die Referenzen in Unity nicht verloren gehen. Gleiches gilt für **Tilesets** der Weltkarte und Objekte (Bäume, Häuser etc.). - Sound: Es gibt eine Handvoll Soundeffekte (für Benachrichtigungen vom Digivice, für Button-Klicks, etc.) und Hintergrundmusik pro Gebiet. Achte darauf, diese entsprechend wieder einzubinden (im Projekt liegen sie unter `Assets/Audio`). Im Inspector der jeweiligen Szenen sind AudioListener und AudioSource mit Referenzen auf diese Dateien vorhanden – stelle sicher, dass nach dem Nachbau diese Referenzen passen.

Steuerung: Das Spiel ist für Touch-Eingaben ausgelegt. Die Implementierung nutzt Unitys **Input-System** für Mobile: - Der Spielercharakter wird entweder per **On-Screen-Joystick** (virtuelles Gamepad) bewegt **oder** per Tap-to-Move (kann in Einstellungen umgeschaltet werden). In der aktuellen Version ist der On-Screen-Joystick als Standard aktiv. - Wichtige Aktionen (Interagieren mit Objekten, Menü öffnen) haben dedizierte Buttons im UI. Beispielsweise erscheint ein „Interagieren“-Knopf, wenn man in

Reichweite eines interessanten Objekts ist. - Das Digivice-Menü und andere Interfaces sind für typische Handy-Bedienung ausgelegt (Scroll-Listen, Touch-Buttons mit ausreichend großer Fläche etc.).

Gameplay und Features

Spielgenre und Ziel: Unser Handyspiel ist im Kern eine **Adventure-/RPG-Mischung** mit leichten Puzzle- und AR-Elementen. Der Spieler schlüpft in die Rolle eines jungen Abenteurers, der mithilfe von Najika (KI-Begleiter) und dem Digivice mysteriöse Ereignisse in seiner Umgebung erkundet. Es gibt eine durchgehende Storyline mit Quests, Nebenaufgaben und sammelbaren digitalen Kreaturen (inspiriert von Digimon). Das Endziel in der Testversion ist es, alle Hinweise zu sammeln, um den Standort der realen „Schwarzen Mühle“ aufzudecken und einen dort lauernden Gegner zu besiegen.

Wichtige Features im Überblick:

- 1. Story und Dialoge:** Die Handlung wird durch Dialoge mit Najika und NPCs vorangetrieben. Dialoge erscheinen als Textblöcke, manchmal mit Auswahlmöglichkeiten für den Spieler (simple Entscheidungen, die den Dialogverlauf beeinflussen). Diese Entscheidungen sind momentan nicht spielentscheidend (kein verzweigtes Storyende), aber sie ermöglichen ein Rollenspiel-Gefühl. Für den Nachbau: implementiere ein **Dialog-System** (z.B. scriptable objects oder eine Dialog-JSON), welches Charaktername, Portrait und Text verwaltet und optional Verzweigungen zulässt. Die KI Najika liefert viele Dialoge dynamisch, aber für einige Story-Kerngespräche wurden festgeschriebene Texte verwendet (insbesondere um die Haupthandlung kontrolliert zu vermitteln).
- 2. Quests und Aufgaben:** Es gibt ein **Quest-System** mit Haupträtseln und optionalen Nebenaufgaben. Beispiele: „Finde im Wald den versteckten digitalen Code“ – das war die AR-Scan-Aufgabe, bei der man draußen einen echten QR-Code suchen und scannen musste, um im Spiel einen Code zu erhalten. Oder „Besiege das abtrünnige Digimon im alten Sägewerk“ – eine Kampfaufgabe, die auf einem kleinen runden **Rasterspielfeld** (ähnlich einem einfachen rundenbasierten Kampfsystem) ausgetragen wird. Für Reproduktion: Stelle die **Questlogik** nach – z.B. als simple Zustandsautomaten: Quest X hat Status {nicht begonnen, aktiv, abgeschlossen}. Trigger (wie der QR-Scan) setzen den Status fort. Das Kampfsystem im Prototyp ist relativ simpel (Rundenkampf mit Auswahl „Angreifen“ oder „Flüchten“), du kannst es zunächst auch prototypisch halten. Wichtig ist nur, dass die **Sieg/Niederlage-Bedingungen** sauber an die Questlogik zurückmelden (z.B. Quest „besiege Digimon“ wird als erfüllt markiert, wenn Kampf gewonnen).
- 3. Sammlungen (Kreaturen/Items):** Der Spieler kann im Verlauf verschiedene **digitale Kreaturen** finden (ähnlich Pokémon/Digimon) – in der Testversion sind 3 Stück implementiert. Diese haben keinen komplexen AI-Kampfverhalten, sie dienen mehr dem Sammeltrieb und der Story. Außerdem gibt es Items (z.B. „Energie-Kristalle“, die man braucht, um die Digivice-Funktionen aufzuladen). Im **Inventar-System** werden diese Objekte gelistet. Repliziere also eine Inventarverwaltung, wo neue Objekte hinzugefügt und angezeigt werden können. In Unity kann man hierfür ScriptableObjects verwenden, welche die Eigenschaften der Kreaturen/Items definieren (Name, Beschreibung, ggf. Effekte). Die *Digivice*-UI greift dann auf diese Daten zu.
- 4. Multi-User-Design (6 Spieler privat):** Das Spiel ist primär **singleplayer** gestaltet, aber es ist vorgesehen, es mit bis zu **6 bekannten Testern** parallel auszuprobieren. In der aktuellen Phase bedeutet das praktisch: wir stellen bis zu 6 Personen eine APK (Android-Paket) zur Verfügung, die sie auf ihren Geräten installieren. Es handelt sich *nicht* um ein gleichzeitig kooperatives Multiplayer-Spiel – jeder spielt für sich, aber wir wollten das Spiel auf verschiedenen Geräten/Typen getestet haben und ggf. Feedback sammeln. Es gibt jedoch *planerisch* schon Gedanken über zukünftige **Multiplayer-Features**: Zum Beispiel könnten Spieler in einer späteren Version **untereinander Daten tauschen** (Kreaturen handeln, oder gegeneinander antreten). Dafür wäre ein Online-Backend nötig (Server oder Cloud-Service). Momentan ist diese Funktion deaktiviert. Für Nachbau: Fokus liegt auf Singleplayer, aber achte darauf, den Code modular zu halten, sodass ein Networking-System später integrierbar wäre (z.B. Trennung von Spiel-Logik und Datenhaltung, um später Servercalls einzubauen).

Spielbalance und kritische Aspekte: Da alle Features in der Testphase „voll drin“ sind, gibt es ein paar Punkte, die vor einer Veröffentlichung entschärft werden müssten. Zum Beispiel taucht im Spiel direkt der Begriff *Digivice* auf – für private Nutzung kein Problem, öffentlich müsste man den Namen ändern, um Lizenzprobleme zu vermeiden. Ebenso gibt es ein NPC, der stark an einen Digimon-Charakter angelehnt ist. Solche **kritischen Anlehnungen** würden wir selbst vor Release umbenennen oder grafisch verändern (die Daten dafür sind aber dokumentiert, um nichts zu vergessen). Auch sicherheitsrelevante Dinge: In der Testversion ist der Zugang zur *Schwarzen Mühle* mit einem trivialen Passwort geschützt und die Kommunikation mit der KI-API erfolgt unverschlüsselt – beides ist okay für einen geschlossenen Test, aber nicht für Public Release. Diese Notizen sind ebenfalls im Entwicklerbereich festgehalten, damit man es beim Nachbauen und späteren Veröffentlichen nicht übersieht.

Testen und Feinschliff

Da bis zu 6 Personen die App testen, stelle sicher, dass das **Deployment** unkompliziert ist: Wir nutzten für Android z.B. einen einfachen Download-Link oder Google Play Internal Test. Jeder Tester bekam einen eigenen Zugangscode für den gesicherten Bereich (vorab auf ihr Gerät angepasst, um Missbrauch zu vermeiden). Achte beim Nachbau, dass du ähnliches machen kannst – zumindest wissen wir jetzt, dass bis zu 6 verschiedene Gerätetypen (unterschiedliche Android-Versionen) das Spiel installieren und spielen können sollen.

Test-Szenarien, die du unbedingt durchspielen solltest: - **Kompletter Story-Durchlauf:** Von Anfang bis Ende – prüfe, ob alle Quests sich in richtiger Reihenfolge lösen lassen, ob Dialoge kontextpassen (gerade die dynamischen KI-Dialoge müssen Sinn ergeben), und ob das Ende erreichbar ist. - **Abweichendes Verhalten:** Versuche auch „unsinnige“ Dinge – z.B. bevor du Quest A erledigst, schon an Ort B zu gehen. Das Spiel sollte darauf reagieren (meist haben wir Barrieren gesetzt: NPCs oder Najika sagen einem dann, man solle erst etwas anderes tun). - **Digivice-Funktionen:** Teste das Scannen an verschiedenen Lichtverhältnissen und mit falschen/korrekten Markern – wie robust ist die Erkennung? Überprüfe, ob alle gesammelten Kreaturen/Items korrekt im Digivice angezeigt werden. - **Performance und Akkubelastung:** Lass die App auch längere Zeit laufen, um sicherzustellen, dass keine Memory Leaks existieren. Besonders die KI-Abfragen könnten bei schlechter Verbindung hängen – implementiere also Timeout-Mechanismen und teste diese (z.B. was passiert, wenn keine Antwort kommt? In unserem Fall gibt Najika nach 10 Sekunden eine Standard-Antwort „Ich habe dich nicht verstanden, kannst du das wiederholen?“, um Hänger zu kaschieren).

Nachdem all diese Punkte geprüft sind, kannst du sicher sein, dass das Mobile Game inhaltlich komplett und technisch stabil ist. Nun richten wir den Blick auf zukünftige Erweiterungen.

Spätere Pläne für UEFN (Unreal Editor for Fortnite)

In Zukunft ist angedacht, das Spiel bzw. dessen Konzept auch auf die Plattform **UEFN** zu bringen. **Unreal Editor for Fortnite (UEFN)** ermöglicht es, eigene Spiel-Inseln in Fortnite zu gestalten und zu skripten – seit dem Start 2023 wurden bereits über 80.000 solcher Custom-Maps von Spielern veröffentlicht ⁴. Eine Umsetzung unseres Projekts dort würde einige Vorteile bieten, aber auch Anpassungen benötigen.

Zielsetzung der Portierung: Durch UEFN könnte das Spiel einem **größeren Publikum** zugänglich gemacht werden, da Fortnite eine riesige Nutzerbasis hat. Spieler könnten ohne separate App-Installation direkt in Fortnite unsere Erlebnis-Insel betreten. Außerdem übernimmt Epic Games in diesem Fall Hosting, Multiplayer-Infrastruktur etc., was den Aufwand für uns reduziert. Das Projekt

bliebe zunächst als **private Insel** konzipiert (nur auf Einladung der bekannten Tester), kann später aber öffentlich gelistet werden. Ein weiterer Beweggrund: Epic belohnt erfolgreiche UGC-Inhalte finanziell – über 320 Mio. USD wurden an Creator ausgeschüttet ⁴ –, was langfristig zur Refinanzierung unserer Entwicklung beitragen könnte.

Umsetzung in UEFN: Eine 1:1-Portierung ist nicht trivial, da unsere mobile App einige spezifische Features hat (z.B. KI-Integration, AR-Elemente), die so in Fortnite nicht direkt nachbildbar sind. Dennoch lässt sich der Kern adaptieren: - **Spielwelt und Grafiken:** Wir würden die wichtigsten Schauplätze (Wald, Dorf, Schwarze Mühle) als **Fortnite-Insel** nachbauen. UEFN erlaubt den Import eigener 3D-Modelle und Texturen in begrenztem Umfang. Alternativ nutzen wir Fortnite-eigene Bausteine, um eine optisch ähnliche Welt zu kreieren. Der Maßstab und Art der Grafik würde sich natürlich ändern (Fortnite-typischer 3D-Stil), aber die Level-Layouts können wir anlehnen. Zum Glück unterstützt UEFN mittlerweile auch **Modularität und eigene Assets**, sodass z.B. charakteristische Objekte (wie vielleicht ein Mühlengebäude) importiert werden könnten, falls nötig. - **Spielmechaniken:** Fortnite Creative (bzw. UEFN) arbeitet mit dem **Verse-Skriptsystem**. Wir müssen unsere Spielmechaniken – Quests, Sammelobjekte, vielleicht Kämpfe – in Verse neu implementieren. Einige Dinge gehen einfacher: Multiplayer ist ja inherent gegeben und funktioniert out of the box für bis zu 16 Spieler auf einer Insel. Wenn wir also z.B. einen Koop-Ansatz versuchen wollten, ginge das hier leichter. Schwieriger ist die **KI Najika:** Fortnite lässt keine externen API-Aufrufe während des Spiels zu, d.h. wir können Najika dort nicht als echte KI-Instanz betreiben. Stattdessen müssten wir uns behelfen, indem Najika in der Fortnite-Version entweder als **statischer NPC** mit vorgegebenen Dialogen agiert, oder wir die KI-Antworten vorher generieren und als Skript hinterlegen. Ein Kompromiss wäre, Najika in UEFN nur begrenzte Sätze sagen zu lassen (z.B. standardisierte Tipps, die wir aus der App-Version übernommen haben). - **Digivice & HUD:** Die Handy-spezifischen Elemente (Digivice UI, AR-Scan) müssten im Fortnite-Kontext anders gedacht werden. Es gibt dort die Möglichkeit, ein **Custom HUD** zu bauen – wir könnten also ein virtuelles Digivice auf dem Bildschirm darstellen, aber es wäre kosmetisch. Ein Scanner via Kamera fällt weg, jedoch könnten wir Sammelemente anders verstecken (Fortnite hat z.B. Sammel-Items, die man aufsammeln kann; das könnte anstelle eines QR-Scans treten: der Spieler findet ein Objekt im Level). - **Community-Aspekte:** In Fortnite könnte man die 6 Tester gleichzeitig auf die Insel lassen; sie könnten die Quests gemeinsam oder gegeneinander erleben. Das ist ein anderer Spielablauf als solo auf dem Handy. Hier muss noch entschieden werden, ob die UEFN-Version **kooperativ** (alle gegen den Endgegner gemeinsam) oder **kompetitiv** (wer löst die Rätsel zuerst?) sein soll. Für den Nachbau sollte man beide Möglichkeiten offenhalten. In Verse kann man beispielsweise eine Race-Mechanik skripten oder Kooperation belohnen.

Schritte zur Portierung: Sobald das Mobile Game stabil läuft, wäre folgendes Vorgehen geplant, um nach UEFN zu gelangen: 1. **Reduktion auf Kern-Gameplay:** Herausarbeiten, welche Elemente unbedingt übertragen werden müssen (Story, Kern-Quest, ein paar Digimon-ähnliche Kreaturen vielleicht als dekorative Begleiter) und welche entfallen können (AR-Scans entfallen, KI dynamisch entfallen, stattdessen statische Alternative). 2. **Umgebung in UEFN bauen:** Mit dem Unreal Editor die Landschaft erstellen. Assets vorbereiten – entweder Fortnite Standard-Assets nutzen (Terrain, Bäume analog Wald, etc.) oder Custom Meshes importieren, falls etwas sehr Spezifisches gebraucht wird (z.B. unser Mühlengebäude). 3. **Spiellogik in Verse programmieren:** Die Questabläufe, Item-Sammeln, eventuelle Rätsel (z.B. in Fortnite könnte man statt QR-Rätsel ein kleines **Schalter-Puzzle** einbauen, was das gleiche Ergebnis triggert). Hier müssen wir auch die NPC-Dialoge umsetzen. Fortnite hat rudimentäre **Cinematic Sequencer**-Möglichkeiten und NPCs, die Texte anzeigen können. Najika könnte als NPC-Charakter im Startgebiet stehen und dem Spieler automatisiert Erklärungen geben (triggerbar per proximity). Der eigentliche intelligente Austausch entfällt, aber man kann zumindest das Flair erhalten. 4. **Test in Fortnite:** Das Insel-Erlebnis dann im geschlossenen Kreis testen, analog zum Handytest. Wahrscheinlich zeigen sich neue Balance-Themen (etwa da Spieler sich gegenseitig beeinflussen können, was es im Singleplayer nicht gab).

Status und Ausblick: Bisher ist die UEFN-Portierung ein Plan; wir haben noch nicht mit der Umsetzung begonnen. Allerdings wurden bereits Voruntersuchungen gemacht: z.B. haben wir geprüft, welche unserer Ideen technisch im Fortnite-Framework umsetzbar sind. Ergebnis: **Grundsätzlich ja**, aber mit den genannten Abstrichen. Sobald die mobile Version abgeschlossen und intern erfolgreich erprobt ist, kann man diese Portierung angehen. Es bietet sich eventuell an, das Projekt leicht umzubenennen oder anzupassen für Fortnite, um dort auf User-Entdeckung zu setzen (der Name „Najika“ bleibt vermutlich, da dieser charakteristisch und eigenständig ist, aber Begriffe wie Digivice würden wir eventuell vermeiden oder nur in Anspielungen verwenden).

Fazit und Überprüfung der Vollständigkeit

In dieser Übersicht wurden **alle relevanten Details** zu den vier Hauptkategorien des Projekts dargelegt:

- Die KI-Komponente **Najika** (Implementierung, Funktionen, Nachbauanleitung)
- Der **gesicherte Entwicklerbereich „Schwarze Mühle“** und das darin eingebettete **Digivice**-Konzept (Hintergrund, Zweck, technische Umsetzung)
- Das eigentliche **Handyspiel**, inkl. aller Gameplay-Features, technischen Grundlagen und Hinweise zur Reproduktion auf einer Mobilplattform
- Die **zukünftigen Pläne für UEFN**, mit Überlegungen zur Portierung und Anpassung des Spiels an Fortnite's Umgebung

Jeder dieser Punkte wurde sorgfältig ausgearbeitet und mit Beispielen erläutert, sodass nichts Wichtiges fehlt. Bitte gehe diese Dokumentation beim Nachbau Schritt für Schritt durch. Sie dient als **Bauplan**, um das Projekt in jedem Detail nachstellen zu können. Solltest du feststellen, dass irgendwo Informationen fehlen, kannst du anhand der klaren Struktur gezielt nachbessern – nach derzeitigem Stand ist jedoch **alles Nötige erfasst**.

Zur Sicherheit wurde jedes Segment mehrfach überprüft, um sicherzustellen, dass **keine Detailinformation verloren gegangen** ist. Mit diesem Dokument in der Hand sollte es möglich sein, das gesamte Projekt **eins-zu-eins** zu rekonstruieren und zum Laufen zu bringen. Viel Erfolg beim Nachbau!

Quellen: (zur Untermauerung einiger Hintergrundbegriffe)

- Digimon-Hintergrund zum **Digivice** (Funktion als Gerät zur Verbindung von Kind und Digimon)
2 3
- Ursprung des Begriffs „**Schwarze Mühle**“ in der Krabat-Sage (Symbol für verborgene Wissen/Magie) 1
- Erfolg und Möglichkeiten von **UEFN** (Unreal Editor for Fortnite) – bereits über 80.000 von Spielern erstellte Fortnite-Inseln seit 2023 4

1 Die schwarze Mühle (Roman) – Wikipedia
[https://de.wikipedia.org/wiki/Die_schwarze_M%C3%BChle_\(Roman\)](https://de.wikipedia.org/wiki/Die_schwarze_M%C3%BChle_(Roman))

2 3 Digivice | DigimonWiki | Fandom
<https://digimon.fandom.com/wiki/Digivice>

4 Key details about Unreal Engine 5.4, plus 'Epic' plans for UEFN and user-generated content in Fortnite | Game World Observer
<https://gameworldobserver.com/2024/03/20/unreal-engine-5-4-details-uefn-ugc-tools-egs-mobile>